

Travaux dirigés n° 2

Structures et fichiers

Dans les exercices suivants, nous considérons la taille et les alignements suivants (correspondant à *gcc* sous *Ubuntu*, sur une architecture 64 bits) :

Type	Taille	Aligne.	Type	Taille	Aligne.	Type	Taille	Aligne.
char	1o	1o	short	2o	2o	int	4o	4o
long	8o	8o	float	4o	4o	double	8o	8o
long long	8o	8o	long double	16o	16o	pointer	8o	8o

Exercice 1 (Structures en C)

Soient les structures suivantes :

```
typedef struct {
    float a;
    char b;
    char c;
    float d;
} structure1_t;
```

```
typedef struct {
    char a;
    char b;
    double c;
    char d;
} structure2_t;
```

```
typedef struct {
    float a;
    long b;
    long long int c;
    int d;
} structure3_t;
```

1. Calculez leur taille.
2. Où se trouvent les bits de bourrage ?
3. Modifiez les structures en ajoutant des champs de bourrage (de type tableau de caractères).
4. Modifiez les structures pour obtenir une taille minimale et calculez leur nouvelle taille.

Exercice 2 (Un peu plus loin)

Soient les structures suivantes :

```
typedef struct {
    int *a;
    char b[2];
    char c[3];
    long double d;
} structure1_t;
```

```
typedef struct {
    int a;
    structure1_t b[2];
    char c[12];
    int d;
} structure2_t;
```

```
typedef union {
    char a;
    double b;
    char *c;
    short d;
} structure3_t;
```

- 1°) Calculez la taille de ces structures et indiquez où se trouvent les octets de bourrage.
- 2°) Proposez une solution permettant d'afficher les octets utilisés par chaque champ de la première structure. Affichez des '?' pour les octets de bourrage, des 'a' pour le champ 'a', etc..

Exercice 3 (Stéganographie)

Nous souhaitons créer une application permettant de loguer des événements d'applications. Chaque événement est identifié par un code d'application (un entier non signé), représentant l'application créant l'événement, une date (un entier long non signé), un code d'état (un int), une priorité (un entier non signé) et un message (une chaîne de 256 caractères). Les événements sont stockés par ordre chronologique dans le fichier.

- 1°) Proposez une structure pour un événement.
- 2°) Quelle est la taille du fichier contenant 100 événements ?
- 3°) Représentez le contenu d'un fichier contenant plusieurs événements.

La stéganographie est l'art de la dissimulation : son objet est de faire passer inaperçu un message dans un autre message. - Wikipedia

Nous souhaitons exploiter le fichier généré par cette application pour y cacher des informations. Le principe est d'écrire un programme qui prend en argument une chaîne de caractères contenant le message à cacher et qui l'ajoute dans le fichier des contacts. Si aucun message n'est passé, le programme récupère le message présent dans le fichier et l'affiche.

- 4°) À quels endroits peut-on cacher des informations dans le fichier ? Est-il possible de les augmenter en modifiant l'ordre des champs de la structure ?
- 5°) Donnez le code de la procédure qui permet de cacher un message dans le fichier.
- 6°) Donnez le code de la procédure qui permet de récupérer le message depuis le fichier.